Attoparsec
Hackage: https://hackage.haskell.org/package/attoparsec

```
-- Data.Attoparsec.Combinator
try :: Parser i a -> Parser i a
(<?>) :: Parser i a -> String -> Parser i a
choice :: Alternative f => [f a] -> f a
count :: Monad m => Int -> m a -> m [a]
option :: Alternative f => a -> f a -> f a

many1 :: Alternative f => f a -> f [a]
manyTill :: Alternative f => f a -> f b -> f [a]
sepBy :: Alternative f => f a -> f s -> f [a]
sepBy1 :: Alternative f => f a -> f s -> f [a]
skipMany :: Alternative f => f a -> f ()
skipMany1 :: Alternative f => f a -> f ()

many' :: MonadPlus m => m a -> m [a]
many1' :: MonadPlus m => m a -> m [a]
manyTill' :: MonadPlus m => m a -> m b -> m [a]
sepBy' :: MonadPlus m => m a -> m s -> m [a]
sepBy1' :: MonadPlus m => m a -> m s -> m [a]

eitherP :: Alternative f => f a -> f b -> f (Either a b)
feed :: Monoid i => IResult i r -> i -> IResult i r

endOfInput :: forall t. Chunk t => Parser t ()
atEnd :: Chunk t => Parser t Bool
lookAhead :: Parser i a -> Parser i a
```

Data.ByteString.Char8 (strict)

```
-- Data.Attoparsec.ByteString.Char8
type Parser = Parser ByteString                          --
Data.ByteString.Char8 (strict)
type Result = IResult ByteString

parse :: Parser a -> ByteString -> Result a
feed :: Monoid i => IResult i r -> i -> IResult i r
parseOnly :: Parser a -> ByteString -> Either String a
parseWith :: Monad m => m ByteString -> Parser a -> ByteString -> m
(Result a)
parseTest :: Show a => Parser a -> ByteString -> IO ()
maybeResult :: Result r -> Maybe r
eitherResult :: Result r -> Either String r

char :: Char -> Paser Char
char8 :: Char -> Parser Word8
anyChar :: Parser Char
notChar :: Char -> Parser Char
satisfy :: (Char -> Bool) -> Parser Char
```

```
peekChar :: Parser (Maybe Char)
peekChar' :: Parser Char

digit :: Parser Char
letter_iso8859_15 :: Parser Char
letter_ascii :: Parser Char
space :: Parser Char

isDigit :: Char -> Bool                                    -- [0-9]
isDigit_w8 :: Word8 -> Bool
isAlpha_iso8859_15 :: Char -> Bool
isAlpha_ascii :: Char -> Bool                              -- [a-
zA-Z]
isSpace :: Char -> Bool                                     -- ' '
\t \r \n \v \f
isSpace_w8 :: Word8 -> Bool
inClass :: String -> Char -> Bool
notInClass :: String -> Char -> Bool

string :: ByteString -> Parser ByteString
stringCI :: ByteString -> Parser ByteString
skipSpace :: Parser ()                                    --
Data.Char.isSpace i.e. ' ' \t \r \n \v \f
skipWhile :: (Char a -> Bool) -> Parser ByteString
take :: Int -> Parser ByteString
takeWhile :: (Char a -> Bool) -> Parser ByteString
takeWhile1 :: (Char a -> Bool) -> Parser ByteString
takeTill :: (Char a -> Bool) -> Parser ByteString

takeByteString :: Parser ByteString
takeLazyByteString :: Parser ByteString

endOfLine :: Parser ()
isEndOfLine :: Word8 -> Bool                              -- \n or
\r\n
isHorizontalSpace :: Word8 -> Bool                        -- ' ' or
\t

data Number = I !Integer | D !Double
decimal :: Integral a => Parser a
hexadecimal :: (Integral a, Bits a) => Parser a
signed :: Integral a => Parser a -> Parser a
double :: Parser Double
number :: Parser Number
rational :: Fractional a => Parser a
scientific :: Parser Scientific
```

Data.Attoparsec.Internal.Types

```
newtype Parser i a = Parser { … }
```

```
data IResult i r =
    Fail i [String] String
  | Partial (i -> IResult i r)
  | Done i r

type family State i
 type instance State ByteString = Buffer
 type instance State Text = Buffer
newtype Pos = Pos { fromPos :: Int }
data More = Complete | InComplete
type Failure i t a = t -> Pos -> More -> [String] -> String ->
IResult i r
type Success i t a = t -> Pos -> More -> a -> IResult i r

-- Chunk
class Monoid c => Chunk c where
 type ChunkElement c
 nullChunk :: c -> Bool
 pappendChunk :: State c -> c -> State c
 atBufferEnd ::
      c                                       -- proxy
   -> State c
   -> Pos
 bufferElemAt ::
      c                                       -- proxy
   -> Pos
   -> State c
   -> Maybe (ChunkElem c, Int)
 chunkElemToChar ::
      c                                       -- proxy
   -> ChunkElem c
   -> Char

instance Chunk ByteString
instance Chunk Text
```

Attoparsec
Features:
- **incremental parsing**:
e.g.

```
import Data.Attoparsec.ByteString.Char8
import Data.Function

p = string "Hello" :: Parser ByteString
parse p "He"                  -- partial
 & (`feed` "l")               -- next
 & (`feed` "l")               -- next
 & (`feed` "oabc")            -- Done "abc" "Hello" :: Result
ByteString
```

- **backtracking**: if parsing fails, tries next option (if available)
e.g.

```
p = char 'a' <|> char 'b'
parseOnly p "aa"        -- success
parseOnly p "bb"        -- success
parseOnly p "cc"        -- fail
```

Basic

```
import Data.Attoparsec.ByteString.Char8
import Control.Monad
import Control.Applicative

char 'a' :: Parser Char
char 'a' <|> char 'b' :: Parser Char           -- a or b
choice [char 'a', char 'b']                    -- same
optional (char 'a') :: Parser (Maybe Char)     -- a or none
 Control.Applicative.optional
option '!' (char 'a') :: Parser Char           -- a or default !
char 'a' <* skipSpace :: Parser Char           -- ignore right
skipSpace *> char 'a' :: Parser Char           -- ignore left
void (char 'a') :: Parser ()                   -- ignore result
Control.Monad.void

char 'a' <?> "parse a"                         -- description for
debugging
```

```
-- e.g.
ghci> p = ((char 'a' <?> "parse a") *> (char 'b' <?> "parse b")) <?
> "parse ab"
ghci> parseOnly p "c"
Left "parse ab > parse a > a: Failed reading: satisfyWith"
ghci> parseOnly p "ac"
Left "parse ab > parse b > b: Failed reading: satisfyWith"
```

Usages

Example: parse words

```
{-# LANGUAGE OverloadedStrings #-}

import Data.Attoparsec.ByteString.Char
import Control.Applicative

word :: Parser String
word = many1 (satisfy (`elem` ['0'..'z'])) <?> "word"

whitespace :: Parser String
```

```
whitespace = many1 space <?> "spaces"

wordsParser :: Parser [String]
wordsParser = (
  optional whitespace
  >> sepBy1 word whitespace
  <* optional whitespace
  <* endOfInput                 -- no more input after trimming
whitespaces
  ) <?> "words"
```

e.g.

```
parseOnly wordsParser "abc"               -- Right ["abc"]
parseOnly wordsParser "  abc   "          -- Right ["abc"]. Spaces
trimmed
parseOnly wordsParser "ab cd 1 23"        -- Right
["ab", "cd", "1", "23"]
parseOnly wordsParser "#"                 -- Left "words: word:
Failed reading: satisfyWith"
parseOnly wordsParser "123#"              -- Left "words:
endOfInput" i.e. did not consume all
```

Example: incremental parsing
Use **Result** or **feed**

```
{-# LANGUAGE OverloadedStrings #-}

import Data.Attoparsec.ByteString.Char8
import Data.ByteString.Char8 qualified as B
import Data.Function

hello :: Parse String
hello = B.unpack <$> string "Hello World"

result1  :: Result String
result1 = parse hello "He"                      -- partial input
"He"

resultSuccess :: Result String
resultSuccess = result1
 & (`feed` "llo")                               -- more input
 & (`feed` "World abc")                         -- more input
                                                -- Done "abc"
"Hello World" :: Result String

resultSuccess2 :: Result String
resultSuccess2 =                                -- Same manually
i.e. use Partial instead of feed
  let
```

```
    Partial p = result1
    Partial p' = p "llo"
  in  p' "World abc"

resultFail :: Result String
resultFail = feed result1 "a"                    -- Fail "Heabc" []
"string" :: Result String
```

Example: Parse from a stream
Use a **ByteStream** (stream of **strict** ByteStrings) + **streaming-utils** (apply parser on ByteStream)
streaming-bytestring: https://hackage.haskell.org/package/streaming-bytestring
streaming-utils: https://hackage.haskell.org/package/streaming-utils

```
{-# LANGUAGE OverloadedStrings #-}

import Data.Attoparsec.ByteString.Char8
import Data.Attoparsec.ByteString.Streaming as BS
import Data.ByteString qualified as B
import Data.Functor.Of
import Streaming.ByteString

a :: Parser Char
a = char 'a'

stream :: ByteStream m ()
stream = fromStrict "aaaaabbb"

main :: IO ()
main = do
  (Left result, restStream) <- BS.parse (many' a) stream
  (rest :> _) <- toStrict restStream
  putStrLn $ "result: " <> result
  B.putStr $ "rest: " <> rest <> "\n"
```

output

```
result: aaaa
rest: bbb
```

Example: combinator implementations

```
{-# LANGUAGE BangPatterns #-}

import Control.Monad
import Data.Functor
import Data.Function ((&))
```

```haskell
choice :: Alternative f => [f a] -> f a
choice = asum

count :: Alternative f => Int -> f a -> f [a]
count n pa
  | n > 0 = (:) <$> pa <*> count (n - 1) pa
  | n == 0 = pure []
  | otherwise = empty

option :: Alternative f => a -> f a -> f a
option a pa = pa <|> pure a

optional :: Alternative f => f a -> f ()
optional pa = void pa <|> pure ()

many :: Alternative f => f a -> f [a]
many pa = pure [] <|> many1 pa

many1 :: Alternative f => f a -> f [a]
many1 pa = (:) <$> pa <*> many pa

many' :: Alternative m => m a -> m [a]
many' pa = pure [] <|> many1' pa

many1' :: Alternative m => m a -> m [a]
many1' pa = ((\(!x) -> (x :)) <$> pa) <*> many' pa

skipMany :: Alternative f => f a -> f ()
skipMany pa = void $ many pa

skipMany1 :: Alternative f => f a -> f ()
skipMany1 pa = pa *> skipMany pa

sepBy :: Alternative f => f a -> f sep -> f [a]
sepBy pa psep = pure [] <|> sepBy1 pa psep

sepBy1 :: forall f a sep. Alternative f => f a -> f sep -> f [a]
sepBy1 pa psep = loop
  where
    loop :: f [a]
    loop = ((:) <$> pa) <*> (pure [] <|> (psep *> loop))

endBy :: Alternative f => f a -> f sep -> f [a]
endBy pa sep = pure [] <|> endBy1 pa sep

endBy1 :: forall f a sep. Alternative f => f a -> f sep -> f [a]
endBy1 pa psep = loop
  where
    loop :: f [a]
    loop = ((:) <$> pa) <*> (psep *> (pure [] <|> loop))
```

```haskell
-- (op a1 (op a2 (... op an-1 (op an a0))))
chainr :: Alternative f => f a -> f (a -> a -> a) -> a -> f a
chainr pa pop a = pure a <|> chainr1 pa pop

chainr1 :: Alternative f => f a -> f (a -> a -> a) -> f a
chainr1 pa pop = pa <|> loop
  where
    loop = (&) <$> pa <*> pop <*> (pa <|> loop)

-- (op an (op an-1 (... (op a2 (op a1 a0)))))
chainl :: Alternative f => f a -> f (a -> a -> a) -> a -> f a
chainl pa pop a = pure a <|> chainl1 pa pop

chainl1 :: forall f a. Alternative f => f a -> f (a -> a -> a) -> f
a
chainl1 pa pop = pa <|> ((&) <$> pa <*> loop)
  where
    -- note: loop on function instead of value
    -- Can't do something like
    --
    --    loopLeftRecursion :: f a -> f a
    --    loopLeftRecursion v = loopLeftRecursion $ (&) <$> v <*>
pop <*> pa
    --
    -- since this is left recursive
    -- i.e. it calls the loop before parsing a single term and
hence loops infinitely.
    --
    -- We need to parse at least one term before calling the loop.
    -- But this implies we need to defer the operation and return a
function (a -> a) instead of a value directly.
    loop :: f (a -> a)
    loop = apply <$> pop <*> pa <*> (pure id <|> loop)
    apply op a foldUp r = foldUp (op r a)

manyTill :: Alternative f => f a -> f a -> f [a]
manyTill pa pend = ended <|> loop
  where
    ended = pend $> []
    loop = (:) <$> pa <*> (ended <|> loop)
```